Multimedia Signal Processing - 2nd Module

February, 10 2016

1. **Question (8 pts)**

Consider two discrete memoryless sources X and Y that emit symbols from the alphabets $\{a_1, a_2\}$ and $\{b_1, b_2, b_3\}$, respectively. The joint p.m.f. is

	b_1	b_2	b_3
a_1	0.1	0.4	0.1
a_2	0.15	0.1	0.15

(a) Design the Huffman code to encode single symbols of Y.

(b) Compute: (i) the average code length of the obtained Huffman code; (ii) the average code length of the Shannon code, and; (iii) the theoretical bound.

(c) Design the Huffman code that jointly encodes one symbol of X and one symbol of Y.

(d) Compute again: (i) the average code length of the obtained Huffman code; (ii) the average code length of the Shannon code, and; (iii) the theoretical bound.

(e) Compare and comment the results obtained in point (b) and point (d).

(f) Compute the mutual information $I(X;Y)$.

Solution

(a) First, we compute the p.m.f. for the considered symbols:

$$\begin{aligned} p(b_1) =& p(b_1, a_1) + p(b_1, a_2) = 0.25, \\ p(b_2) =& p(b_2, a_1) + p(b_2, a_2) = 0.5, \\ p(b_3) =& p(b_3, a_1) + p(b_3, a_2) = 0.25. \end{aligned}$$

Notice that $p(b_1) + p(b_2) + p(b_3) = 1$. A possible Huffman code is reported in Table 1. The associated Huffman tree is reported in Figure 1.

Symbol	p.m.f.	Code	Length (l_H)
b_1	0.25	11	2
b_2	0.5	0	1
b_3	0.25	10	2

Table 1: Huffman code computed in point (a).

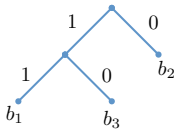


Figure 1: Huffman tree computed in point (a).

(b) The average Huffman code length is $L_H = \sum_{i=1}^{3} p(b_i) l_H(b_i) = 0.25 \times 2 + 0.5 \times 1 + 0.25 \times 2 = 1.5$ bits per symbol. The average Shannon code length is $L_S = \sum_{i=1}^{3} p(b_i) l_S(b_i) = -\sum_{i=1}^{3} p(b_i) \lceil \log_2(p(b_i)) \rceil = 0.25 \times 2 + 0.5 \times 1 + 0.25 \times 2 = 1.5$ bits per symbol. The theoretical bound is given by the entropy $H(Y) = -\sum_{i=1}^{3} p(b_i) \log_2(p(b_i)) = 0.25 \times 2 + 0.5 \times 1 + 0.25 \times 2 = 1.5$ bits per symbol.

(c) The p.m.f. of symbol pairs $\{a_i, b_j\}$, $i \in \{1,2\}$, $j \in \{1,2,3\}$ is reported in the text. Table 2 and Figure 2 report the requested Huffman code and the associated tree.

Symbol	p.m.f.	Code	Length (l_{H_2})
$\{a_1, b_1\}$	0.1	0001	4
$\{a_1, b_2\}$	0.4	1	1
$\{a_1, b_3\}$	0.1	0000	4
$\{a_2, b_1\}$	0.15	010	3
$\{a_2, b_2\}$	0.1	011	3
$\{a_2, b_3\}$	0.15	001	3

Table 2: Huffman code computed in point (c).

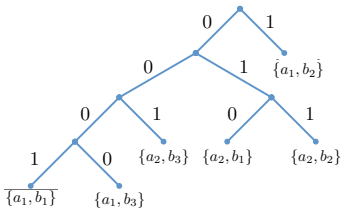


Figure 2: Huffman tree computed in point (c).

(d) Repeating the steps of point (b) we can compute $L_{H_2} = 2.4$, $L_{S_2} = 2.9$ and $H(X,Y) = 2.34$ bits per supersymbol. To compare these results with those obtained in point (b) we need to divide these lengths by 2 (i.e., the number of symbols per supersymbol).

(e) Concerning point (b), the theoretical relationship among the computed quantities is $H(Y) \leq L_H \leq L_S < H(Y) + 1$. More specifically, we notice that $H(X) = L_H = L_S = 1.5$ bits per symbol. This equality holds as the p.m.f. $p(b)$ is dyadic. Concerning point (d), the theoretical relationship is $H(X,Y)/2 \leq L_{H_2}/2 \leq L_{S_2}/2 < H(X,Y)/2 + 1/2$. In this case inequalities hold as the p.m.f. $p(a,b)$ is not dyadic.

(f) We can compute $I(X,Y)$ exploiting the property $I(X,Y) = H(X) + H(Y) - H(X,Y)$. To this purpose, let us compute $H(X) = -\sum_{i=1}^{2} p(a_i)\log_2(p(a_i)) = -0.6\times\log_2(0.6) - 0.4\times\log_2(0.4) = 0.97$. The mutual information is $I(X,Y) = 0.97 + 1.5 - 2.34 = 0.13$.

2. **Question (5 pts)**

(a) Illustrate the block diagram of a closed-loop DPCM coder and explain when it is useful.

(b) Discuss the difference between a closed-loop and open-loop implementation.

Solution See lecture slides.

3. **Question (5 pts)**

(a) Describe the source-system model used to synthesize speech signals.

(b) Explain how a linear predictive vocoder exploits this scheme, and which are the main parameters sent from the encoder to the decoder.

Solution See lecture slides.

4. **Questions (5 pts - each answer can be either TRUE or FALSE)**

(a) Consider a video coding scheme:

- [T] [F] Motion estimation is performed in the KLT domain.
  FALSE. It is performed in the pixel domain (i.e., blocks of pixels are matched, not blocks of KLT coefficients).
- [T] [F] Motion vectors are estimated at the encoder and transmitted to the decoder.
  TRUE.
- [T] [F] Motion vectors can be estimated with different algorithms, all giving the same result.
  FALSE. Different algorithms provide different results (e.g., spiral-search does not provide the same MVs obtained with full-search). There is always a trade-off between computational complexity and MV estimation accuracy.
- [T] [F] Motion estimation can be applied also on blocks whose size is different from 8×8 pixels.
  TRUE.

(b) Consider perceptual audio coding schemes:

- [T] [F] Both tonal and noise maskers are considered by the MPEG-1 psychoacoustic model.
  TRUE.
- [T] [F] MDCT transform is not critically sampled as M coefficients are generated from each $2M$ samples.
  FALSE. MDCT is critically sampled.

(c) Consider an image coding scheme:

- [T] [F] The YCbCr color-space does not provide any advantage over the RGB one.
  FALSE. Changing colorspace may reduce entropy. Moreover, as human visual system is more sensitive to luminance, changing colorspace allows to better exploit visual irrelevancies effects (e.g., sub-sample Cb and Cr).
- [T] [F] The 2D DFT transform is separable.
  TRUE.

(d) Consider a continuous random variable X with uniform p.d.f. and standard deviation σ_X:

- [T] [F] For quantization, the use of compander-expander is preferable to the use of a scalar uniform quantizer.
  FALSE. If the distribution is uniform, there is no reason for using a compander-expander in scalar quantization. The scalar uniform quantizer is already the best we can use.
- [T] [F] According to rate-distortion theory, $R(D) = 0$ for $D = \sigma_X^2$.
  TRUE.

5. **Question (Matlab - 10pts)**

(a) Read the image *image1.png*. Convert it from the RGB to the YCbCr color space according to the transformation

$$\begin{bmatrix} x_Y \\ x_{Cb} \\ x_{Cr} \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.169 & -0.331 & 0.5 \\ 0.5 & -0.419 & -0.0813 \end{bmatrix} \begin{bmatrix} x_R \\ x_G \\ x_B \end{bmatrix}$$

(b) Considering only the Y component, implement a block-wise transform coding scheme based on 2D DCT. Repeat the coding step six times using each time a different block-size and quantization-step: (i) 8×8 and 16×16 pixels blocks; (ii) quantization steps $\Delta \in \{2, 11, 20\}$, paying attention to use a midtread quantizer.

(c) Plot two rate-distortion (RD) curves, i.e., one for each different block size. Use PSNR as distortion metric and assume you are able to encode quantized coefficients with the minimum amount of bits theoretically possible.

(d) Repeat the same steps for the file *image2.png*. Compare RD curves of *image1.png* with those of *image2.png*. Comment what you see (hint: have a look at the histograms of DCT coefficients before and after quantization).

Solution

```
clc, clear all, close all


%% Parameters
%%% Considered block sizes
B_all = [8 16];

%%% Considered quantization steps
delta_all = [2 11 20];

%%% Considered images
img_file_all = ['image1.png'; ...
    'image2.png'];


%% Repeat everything for each image
for i = 1:size(img_file_all,1);

    %% Load image
    img_file = img_file_all(i,:);
    img = double(imread(img_file));


    %% Convert to Y Cb Cr
    R = img(:,:,1);
    G = img(:,:,2);
    B = img(:,:,3);

    Y = 0.299*R + 0.587*G + 0.114*B;
    Cb = -0.169*R -0.331*G + 0.5*B;
    Cr = 0.5*R -0.419*G -0.0813*B;

    clear R G B Cb Cr % free memory


    %% Block-wise transform coding
    %%% Initialize variables
    psnr = zeros(length(B_all), length(delta_all));
    H = zeros(length(B_all), length(delta_all));

    for d = 1:length(delta_all) % loop over delta steps
        %%% Select delta
        delta = delta_all(d);

        for b = 1:length(B_all) % loop over block sizes
            %%% Select block size
            B = B_all(b);

            %%% Display advancment information
            fprintf('Coding %s [delta=%d, block-size=%dx%d]\n', img_file, delta, B, B);

            %%% Initialize reconstructed image
            Y_hat = zeros(size(Y));

            %%% Initialize coefficients
            coeffs = zeros(size(Y));
            coeffs_q = zeros(size(Y));

            %%% Block-wise processing
            for r = 1:B:size(Y,1)-B+1
                for c = 1:B:size(Y,2)-B+1

                    %%% Extract block
                    block = Y(r:r+B-1, c:c+B-1);

                    %%% Compute DCT
                    block_dct = dct2(block);
                    coeffs(r:r+B-1, c:c+B-1) = block_dct;
```

```
                    %%% Quantize DCT coefficients
                    block_dct_q = delta*round(block_dct/delta);
                    coeffs_q(r:r+B-1, c:c+B-1) = block_dct_q;

                    %%% Reconstruct block
                    block_hat = idct2(block_dct_q);
                    Y_hat(r:r+B-1, c:c+B-1) = block_hat;

                end
            end

            %%% Compute PSNR
            mse = mean( (Y(:)-Y_hat(:)).^2 );
            psnr(b,d) = 20*log10(255./sqrt(mse));

            %%% Compute Rate (Entropy of quantized DCT coefficients)
            bins = min(coeffs_q(:)):1:max(coeffs_q(:));
            h = hist(coeffs_q(:), bins);
            h(h==0) = [];
            p = h/sum(h);
            H(b,d) = -sum(p.*log2(p));

            %%% Save coefficients in two cases for late analysis
            if i==1 && B==8 && delta==11 % image1.png, B=8, delta=11
                coeffs_1 = coeffs;
                coeffs_q_1 = coeffs_q;
            elseif i==2 && B==8 && delta==11 % image2.png, B=8, delta=11
                coeffs_2 = coeffs;
                coeffs_q_2 = coeffs_q;
            end

        end

    end


    %% Display result
    figure(i), hold on
    title(sprintf('Results for %s', img_file))
    plot(H(1,:), psnr(1,:), 'linewidth', 2)
    plot(H(2,:), psnr(2,:), 'linewidth', 2)
    xlabel('Rate [bps]'), ylabel('PSNR [dB]')
    axis tight, grid on
    legend('Block: 8x8', 'Block: 16x16', 'Location', 'SouthEast')
    set(gca,'FontSize',14)

end

%%% Comment:
% Rate-distortion curves show the typical 6dB/bit behaviour. The
% only curve that differs from the others is the one obtained for
% image2.png and block size 8x8. In particular, a spike can be
% observed for delta=11. Let us compare DCT coefficients of image1.png
% and image2.png obtained for delta=11 and B=8!


%% Display some DCT coefficients before and after quantization
figure(3)

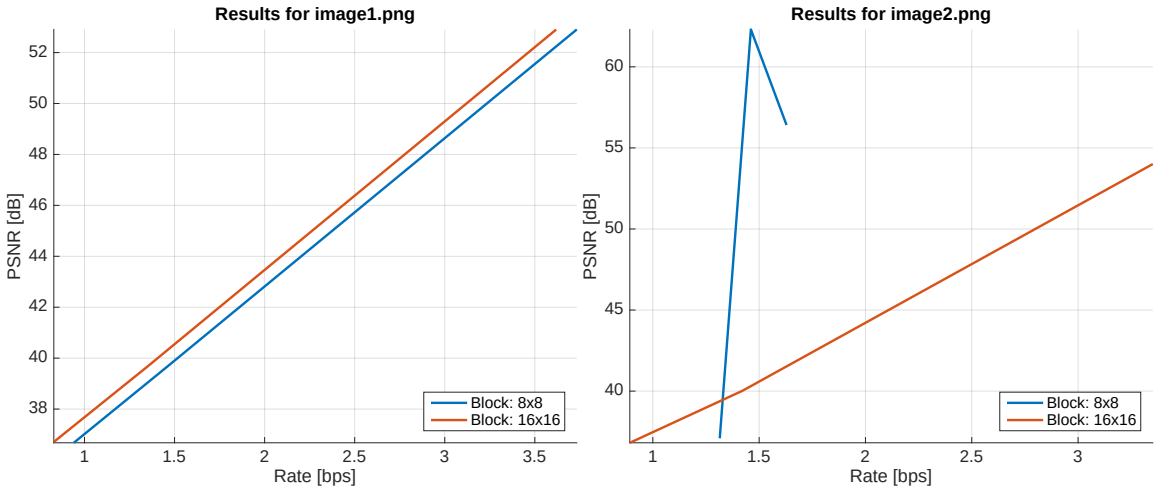
subplot(221), hist(coeffs_1(:), min(coeffs_1(:)):max(coeffs_1(:)));
title('image1.png - before quantization')
xlim([-50 50]), xlabel('DCT coefficient values'), ylabel('Occurrencies')

subplot(222), hist(coeffs_q_1(:), min(coeffs_q_1(:)):max(coeffs_q_1(:)));
title('image1.png - after quantization')
xlim([-50 50]), xlabel('DCT coefficient values'), ylabel('Occurrencies')

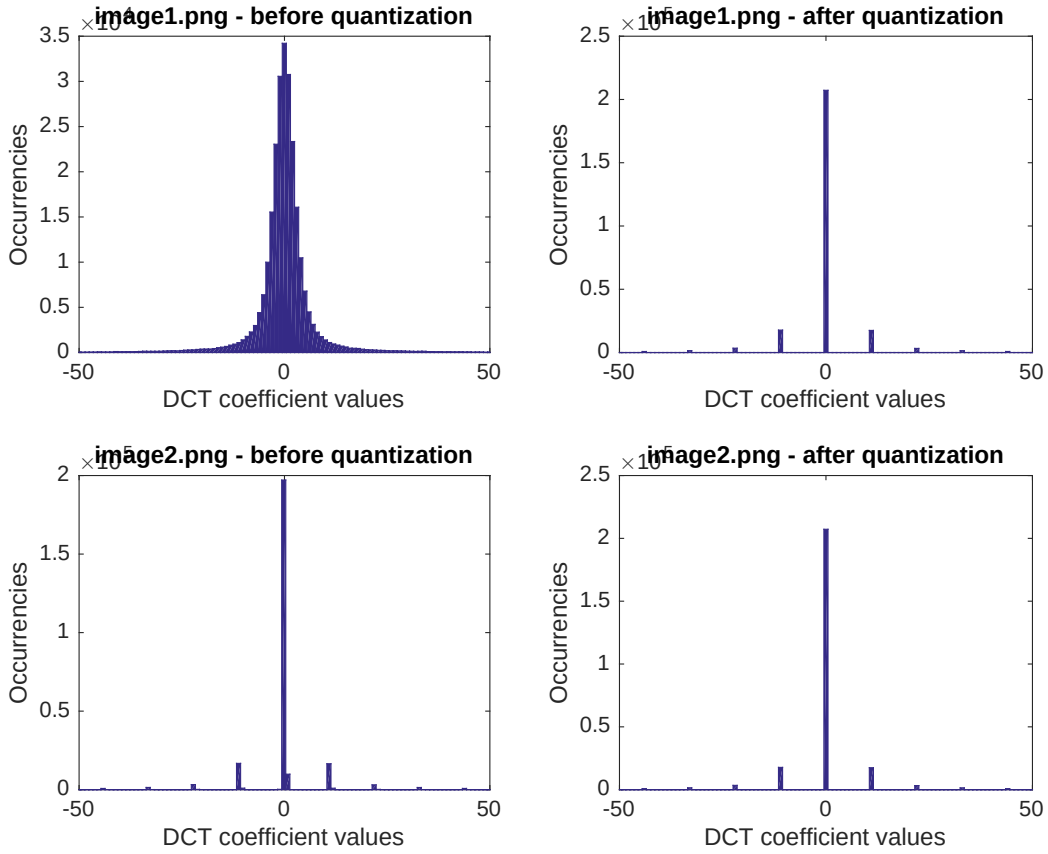
subplot(223), hist(coeffs_2(:), min(coeffs_2(:)):max(coeffs_2(:)));
title('image2.png - before quantization')
xlim([-50 50]), xlabel('DCT coefficient values'), ylabel('Occurrencies')

subplot(224), hist(coeffs_q_2(:), min(coeffs_q_2(:)):max(coeffs_q_2(:)));
title('image2.png - after quantization')
xlim([-50 50]), xlabel('DCT coefficient values'), ylabel('Occurrencies')

%%% Comment:
% Coefficients of image2.png have been previously quantized (see
% subplot(223)). For this reason, the rate distortion curve has a peak at
% delta=11. We are basically re-applying quantization with the same delta,
% thus obtaining Y_hat ~= Y
```


Results for image1.png
PSNR [dB]
Rate [bps]
Block: 8x8
Block: 16x16
Results for image2.png
PSNR [dB]
Rate [bps]
Block: 8x8
Block: 16x16

(a) (b)


image1.png - before quantization
Occurrencies
DCT coefficient values
image1.png - after quantization
Occurrencies
DCT coefficient values
image2.png - before quantization
Occurrencies
DCT coefficient values
image2.png - after quantization
Occurrencies
DCT coefficient values

(c)